Multicommodity Prices Prediction Using Multivariate Data-Driven Modeling: Indonesia Case

Marsellino Prawiro Halim, Novanto Yudistira, and Candra Dewi

***Abstract*— One of the problems experienced by micro, small, and medium enterprises (MSMEs) during this pandemic is that most MSME actors do not understand plan-making during a crisis. This situation was exacerbated by erratic commodity prices, which resulted in several MSME players choosing to temporarily close because their turnover got a drastic decline. To help MSME actors maintain their business by knowing commodity price predictions, we propose a deep learning model using the long short-term memory (LSTM) method to predict commodity prices in Indonesia. LSTM is a type of recurrent neural network (RNN) with a memory cell to store information and solve the vanishing gradient problem in RNN. Furthermore, multivariate LSTM leverages the model to predict datasets with more than one feature. This study used a dataset collected from the Pusat Informasi Harga Pangan Strategis Nasional (PIHPS Nasional) managed by the Indonesian Ministry of Finance and Bank Indonesia consisting of significantly contributed food commodities to the formation of (strategic) inflation rates in Indonesia. The time range of commodity prices is from August 1, 2017, to July 30, 2021. There are 11 commodity price features in the dataset, namely, rice, chicken meat, eggs, onions, garlic, large red chilies, curly red chilies, red chilies, green chilies, cooking oil, and sugar. The lowest mean absolute error (MAE) on prediction is up to 255.998 obtained by the attention multivariate LSTM model with the Adam optimizer, adding batch normalization (Batchnorm) layer, reducing LSTM layer, hidden size, and grouped features. It makes the prediction more accurate and avoids overfitting and underfitting in this case.**

***Index Terms*— Commodity prices, data-driven, deep learning, long short-term memory (LSTM), multivariate, time series prediction.**

I. Introduction

THE current development of the digital economy can result in setbacks, even bankruptcy for micro, small, and medium enterprises (MSMEs) actors who still use conventional systems. The Covid-19 pandemic has exacerbated this situation. The Covid-19 pandemic since 2020 had a significant impact on the MSME business. As many as 96% of MSME actors in Indonesia experienced a negative impact from the Covid-19 pandemic on their business continuity. Meanwhile, 75% of MSME actors experienced a decrease in sales. Moreover, 51% of the MSME actors predicted that their business could last only three months at most. In addition, 67% of the MSME actors felt uncertainty in obtaining emergency funds. Finally, 75% of the MSME actors do not know about policy-making during a crisis like the current one. In contrast, only 13% of the MSME actors are sure that they have plans and solutions related to the crisis period for business resilience [1].

Erratic commodity prices can also disturb the stability of MSMEs. The prices of commodities increased unexpectedly in January 2021. For example, red chilies, which have a regular Rp. 40 000/kg, had touched Rp. 100 000/kg, as well as other commodities that impact MSME actors. Some of them even choose to temporarily close because the turnover has fallen by 30%–50% [2]. Meanwhile, MSMEs are an important sector for economic growth in Indonesia. The number of MSMEs in Indonesia in 2018 reached 64.2 million and is predicted to continue to increase this year. One form of the contribution of MSMEs is that the average contribution to the gross domestic product (GDP) of the Indonesian state reaches more than 50% during 2017–2020 [1]. This contribution encourages the economy of Indonesia. Therefore, we can conclude several challenges and difficulties faced in this work as follows.

1) The commodity highly fluctuates and is uncertain, making accurate predictions hard to be achieved.
2) The price of one commodity and the others might be correlated and mutual information between them potentially exists in increasing prediction accuracy. To confirm this requires a multivariate analysis approach to evaluate multicommodity price prediction.
3) The suitable model with proper hyperparameter setting needs to be explored since the availability of data is limited and overfitting potentially occurs.

To address challenges and difficulties, we leverage the long short-term memory (LSTM) model to train commodity price data. LSTM consists of several activations of forget, input, and output gates to capture the long time series patterns. Several studies show that LSTM performs the best among evaluated methods such as auto-regressive integrated moving average (ARIMA) and recurrent neural network (RNN) for predictions [3], [4], [5]. Time series data are dynamic, and sometimes predictable patterns are captured in a long sequence. In that case, LSTM captures long sequences very well compared with ARIMA and RNN by avoiding vanishing gradients. However, time series data have different characteristics from other modalities, such as text. In the text, there is a need to understand the context more comprehensively by mixing forward and backward propagation. This mixed propagation is represented in the form of Bi-LSTM. Another example,

Manuscript received 18 July 2022; revised 24 November 2022 and 4 December 2022; accepted 6 December 2022. Date of publication 26 December 2022; date of current version 3 December 2024. *(Corresponding author: Novanto Yudistira.)*

Marsellino Prawiro Halim is with Bank Mandiri, Jakarta 10310, Indonesia.

Novanto Yudistira and Candra Dewi are with the Department of Informatics Engineering, Faculty of Computer Science, University of Brawijaya, Malang, East Java 65145, Indonesia (e-mail: yudistira@ub.ac.id).

Digital Object Identifier 10.1109/TCSS.2022.3229772

2329-924X © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Universitas Diponegoro (UNDIP). Downloaded on May 07,2026 at 15:25:51 UTC from IEEE Xplore. Restrictions apply.

Transformer network [6], which performs well in predicting the virus pandemic, produces poor results in price prediction compared with LSTM in this experiment. It proves that each knowledge domain has a unique pattern from which a proper model needs to be discovered.

To help MSME actors have plans and solutions to maintain their business by knowing commodity price predictions, the author will create a machine learning model with the LSTM method to predict commodity prices in Indonesia. LSTM is an RNN with a memory cell. Thus, it stores information and solves the vanishing gradient problem in RNN [7].

Since RNN uses the hyperbolic tangent with a derivative between 0 and 1, if the derivative is too little, it causes a vanishing gradient. Therefore, the neural network cannot be trained due to a minimal gradient. However, the LSTM network can overcome drawbacks in the RNN [3]. Inside the LSTM are the input gates, forget gates, and output gates used for modeling spatiotemporal relationships and dynamic multivariate time series data [8].

Several studies have been conducted, including research by Makhlouf and Malhotra [9], which predicts stock market prices using the LSTM method. Moreover, some studies conducted by Song et al. [10] found that the LSTM method had a better ability than artificial neural network (ANN) and RNN in predicting the time series. Time series prediction research was also carried out by Fischer and Krauss [11], which obtained the results that the accuracy of the LSTM method was better than the random forest, deep neural network, and logistic regression classifier methods.

Based on the background and several journals mentioned above, the LSTM method has effectively solved the time series prediction problems. Therefore, the authors decided to research commodity price time series predictions in Indonesia using the LSTM method. The dataset used in this study is obtained from Pusat Informasi Harga Pangan Strategis Nasional (PIHPS Nasional), a site managed by the Indonesian Ministry of Finance and Bank Indonesia that presented food commodities with a significant contribution to the formation of (strategic) inflation rates in Indonesia. For model evaluation, we use the mean absolute error (MAE). MAE is the average absolute error between the actual value and the predicted value [12]. This study will focus on finding the best configurations and hyperparameters of the LSTM method. Thus, it is hoped that the MAE of the prediction results will be as low as possible.

Thus, our contribution can be summarized as follows.

1) We show that the variant of LSTM, namely, Bi-LSTM, Conv LSTM, and Conv Bi-LSTM, performs poorly in the case of price prediction compared with the standard LSTM.
2) We conclude that the standard LSTM suitably copes with the problem in time series of commodity price due to its ability to overcome the limitation of data and thus avoids overfitting.
3) We show that the multivariate LSTM outperforms univariate one in price prediction showing the significance of multivariate analysis. On the other hand, it shows the existence of mutual information and correlation among commodity prices.

II. Literature Review

A. Related Work

Many studies have proven that using the LSTM method has good results in predicting the long time series. Some of these studies will be discussed in the following subchapter.

The first study that will be discussed in this subchapter was the research conducted by Song et al. [10]. The researchers compared several methods, namely, ANN, RNN, decline analysis, and LSTM, to solve the time-series-based problems on the oil production dataset in Xinjiang, China. The MAE that the researchers got for the ANN was 3.46, RNN was 3.18, decline analysis was 2.18, and LSTM was 1.60 (the lower the MAE, the better the result). From these results, the LSTM method is better for solving the time series problems than ANN, RNN, and decline analysis.

The second study that will be discussed is the research on the time series problems conducted by Xingwei and Kuniaki [13]. In this study, the researchers compared the LSTM method with the ARIMA to solve highway traffic flow datasets. The study found that ARIMA's root mean squared error (RMSE) in peak season, off-season, holiday, and non-holiday was 666.05, 470.21, 1062.01, and 473.23, respectively. Meanwhile, the RMSE in peak season, off-season, holiday, and non-holiday for LSTM was 594.58, 513.33, 844.14, and 485.07, respectively. The researcher's conclusion from this study was that LSTM got lower RMSE results than ARIMA in peak season and holidays, which means LSTM has a better performance than ARIMA in conditions of sharp fluctuations.

The last study that will be discussed is the research by Arfan and Etp [14]. In this study, the researchers compare LSTM with support vector regression (SVR) on stock price datasets in Indonesia. In this study, the mean squared error (MSE) results in one day, 30 days, and 60 days for LSTM were 0.0013, 0.0016, and 0.0013, respectively. Meanwhile, the MSE for SVR in periods of one day, 30 days, and 60 days was 0.0025, 0.0045, and 0.0065, respectively. Therefore, it concludes that LSTM is better than SVR in stock price prediction.

B. Time Series Prediction

Time series prediction is a method to explore and analyze data in time series collected in a specific period. This technique is used to produce a prediction of the time series in the future for arbitrary periods. For example, it predicts stock prices, store product sales, weather, and population in an area [15].

C. Commodity Prices in Indonesia

Several factors influence commodity price fluctuations in Indonesia: supply and demand in the market, input prices, community product prices, the weather from Indonesia, and the increase in fuel price [16]. Community product prices come from commodities such as chicken price fodder. Input prices used to produce these commodities are plant seeds, fertilizers, labor, and equipment. Meanwhile, fuels are used in vehicles to distribute these commodities. In addition, the condition of the Covid-19 pandemic also affects commodity price fluctuations. The rules regarding large-scale social restrictions

Authorized licensed use limited to: Universitas Diponegoro (UNDIP). Downloaded on May 07,2026 at 15:25:51 UTC from IEEE Xplore. Restrictions apply.

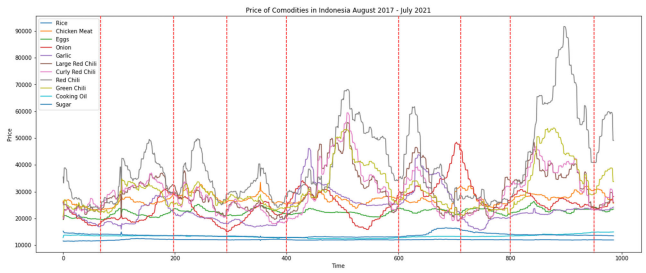


Fig. 1. Seasonality of commodity prices.

and the enforcement of restrictions on community activities implemented by the government as a solution to the Covid-19 pandemic in Indonesia affect the decline in the number of buyers caused by the decline in people's purchasing power and lack of availability in the market caused by by-product buildup [17].

Fig. 1 shows a graph of commodity prices taken from the National PIHPS [18]. It also includes the dynamic seasonal rise and fall over time with different amplitudes. The trend reveals that each rising commodity price season occurs around 100 days long, except 200 days in 2018 over the x-axis, starting from 400th day to 600th day. The rising commodity price season occurs every March to November, with the peak in June 2019. During the increasing price season, there is the holy month of Ramadhan, which is celebrated by the majority of the Indonesian population who are Muslim, so there is a high demand for commodities, leading to an increase in commodity prices.

D. Deep Learning

Deep learning is part of machine learning; however, there are characteristics of deep learning that are not available in general machine learning. One of the prominent characteristics is the model can increase the number of layers used to train from a data representation to produce meaningful information. In deep learning, the number of layers used in training is called depth [19].

1) Long Short-Term Memory (LSTM): LSTM is an RNN and deep learning technique proposed by Hochreiter and Schimdhuber in 1997 to solve long-term time series problems. The LSTM structure consists of an input, forget, update, and output gate. This structure can overcome the problem of vanishing and exploding gradients in the traditional RNN [20].

Fig. 2 illustrates the LSTM structure, which has four gates to process the input data into the output data. The steps of data processing in LSTM are as follows [21]:

$$f_t = \sigma\left(W_{\text{if}}x_t + b_{\text{if}} + W_{hf}h_{t-1} + b_{hf}\right) \quad (1)$$

$$i_t = \sigma(W_{ii}x_t + b_{ii} + W_{hi}h_{t-1} + b_{hi}) \quad (2)$$

$$o_t = \sigma(W_{io}x_t + b_{io} + W_{ho}h_{t-1} + b_{ho}) \quad (3)$$

$$g_t = \tanh\left(W_{ig}x_t + b_{ig} + W_{hg}h_{t-1} + b_{hg}\right) \quad (4)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \quad (5)$$

$$h_t = o_t \odot \tanh(c_t) \quad (6)$$

where f_t, i_t, o_t, and g_t are the forget, input, output, and cell state candidate gate, respectively, while c_t and h_t are the cells

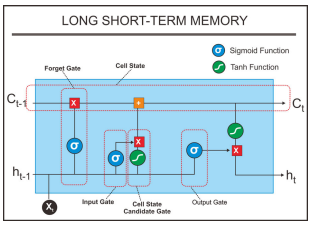


Fig. 2. LSTM structure.

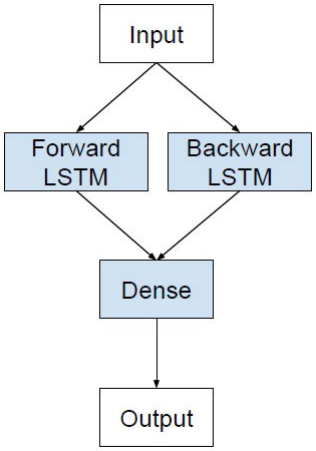


Fig. 3. Bidirectional LSTM architecture.

and hidden state update of the respective tth time. x_t is the input of the respective tth time, and h_{t-1} is the initial, t-1, or hidden state from the previous layer, whereas W and b are the weight and bias of the respective gate. σ is the sigmoid function, $\odot$ is the Hadamard product, and tanh is tan hyperbolic function.

2) Stacked LSTM: Stacked LSTM is one method to add depth to the LSTM model used. It is generally good to add depth to an LSTM model if the resulting model is not overfitted. Some points regarding the use of stacked LSTM are as follows [19].

1) As long as it is not overfitting, the depth of the LSTM model can be added. Of course, this will increase the computational time required to train the model.
2) The results of increasing the depth of the LSTM model may be worse. Because adding layers is not helped by several significant factors.

3) Bidirectional LSTM: Bidirectional LSTM is a modification of the baseline LSTM. The fundamental difference between LSTM and bidirectional LSTM is that for LSTM, the data will be processed in the order. In contrast, for the bidirectional LSTM, the data will be processed in two directions: 1) according and 2) reverse to the sequence.

Fig. 3 is an architecture of a bidirectional LSTM that explains the works of adding an LSTM layer, reversing the sequence, and processing the data—then collecting information both sequentially and in reverse order. Furthermore, the reverse order of the sequence contradicts the philosophy that theoretically, the time-series-based data should be presented in the order of the sequences. One example of a good use case for bidirectional LSTM is speech recognition or human speech because there is evidence that the context of the entire human

Authorized licensed use limited to: Universitas Diponegoro (UNDIP). Downloaded on May 07,2026 at 15:25:51 UTC from IEEE Xplore. Restrictions apply.

speech can be understood better than linear interpretation alone [22].

4) Convolutional LSTM: Convolutional neural network LSTM (CNN-LSTM) is an LSTM layer with a CNN layer added to the model configuration. The use of convolutional neural network (CNN) in the LSTM model configuration is used to perform feature extraction from the input data, combined with LSTM, which is used to make predictions in sequence or time series. Specifically, CNN-LSTM is intended for image description, video, and activity recognition problems. The use of the LSTM architecture and configuration is suitable for problems with the following characteristics [22].

1) Has spatial structure, such as pixels in pictures, and the arrangement of words, such as sentences and paragraphs.
2) Has temporal structure such as the order of frames in the video and the order of words in sentences.

5) Batch Normalization Layer: Batch normalization (Batchnorm) is a layer that optimizes and accelerates the training of deep learning models. In deep learning model training, the change in parameters from the previous layer will affect the distribution of the next layer. The change causes the training to be slow because it requires a lower learning rate. This phenomenon is known as internal covariate shift (ICS). The batch norm layer works by independently standardizing the input data for each scalar feature by making it have a mean of 0 and a variance of 1. The use of the batch norm layer has advantages, including the model can use a higher learning rate to reach the convergence point faster without worrying about exploding and vanishing gradients. The convergence can be achieved because the batch norm layer normalizes activation across the model network, preventing small changes in the parameters and turning them into significant and optimal changes in activation on the gradient. This can prevent the training model from being trapped in wrong or small gradient values. In addition, using this layer can also make the model no longer produce a deterministic value. This significantly affects the generalization of the network in the model. Of course, when the model is more general, the overfitting can be minimized [23]. The equation used in the batch norm layer is [24]

$$y = \frac{x - E[x]}{\sqrt{\text{Var}[x] + \epsilon}} * \gamma + \beta \tag{7}$$

where y is the result, x is the input data, $E[x]$ is the mean of the input data, $\text{Var}[x]$ is the variance of the input data, ϵ is a small number for numerical stability, whereas γ and β are the learning parameters for data scaling and shifting, respectively.

E. Optimization

Optimization is a method used to minimize or maximize (optimize) a function used in deep learning. The function to be optimized is called the objective or criterion function. When the function is minimized, it is called a loss or error function. The loss function is obtained from the derivative of a function to be optimized. The function of getting the derived value is that the deep learning model can determine how much a variable in the function must be changed to match the results obtained. There are several algorithms to perform optimization (called optimizer), including stochastic gradient descent (SGD) and Adam [25].

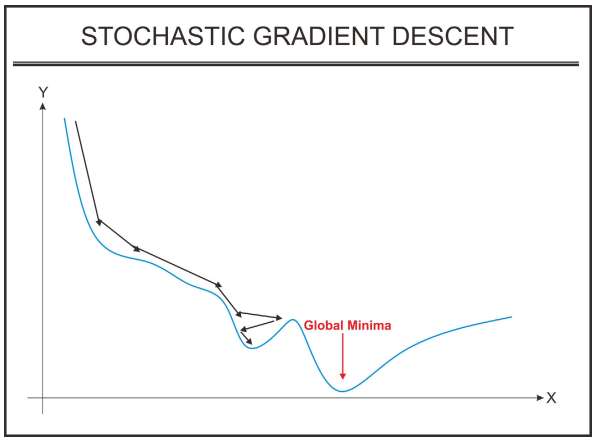


Fig. 4. SGD.

1) Stochastic Gradient Descent (SGD): The weights on an ANN of machine learning and deep learning cannot be calculated using analytical calculations but must be calculated using the empirical calculation procedures. One of the empirical calculation methods that can be used is the SGD [26]. SGD is an optimization method in large-scale machine learning that has variable performance depending on the selected steps [27].

Fig. 4 is an optimization process in machine learning using the SGD method. The SGD method will update the weight and bias parameters on the machine learning model in the back-propagation phase using the gradient value obtained through the partial derivative of a function. So that the best weight and bias parameters will be obtained, which are the minimum global values for the function. The equation used in SGD is

$$\theta = \theta_{t-1} - (\alpha * \text{grad}) \tag{8}$$

where θ_{t-1} is the old parameters, whereas θ, α, and grad are the updated parameters, learning rate, and gradient of the criterion, respectively.

2) Adam: Adam is one of the optimization methods based on adaptive learning rate optimization. Adam is designed by combining the advantages of two well-known optimization algorithms, AdaGrad and RMSProp. Thus, Adam can work well with sparse gradients (an advantage over AdaGrad) and adjust weights to the latest average gradient magnitude, which can work well in a nonstationary (noisy) state. The sparse gradient is a condition where the deep learning model does not receive a strong signal to adjust its weight. One example is the vanishing gradient condition in the RNN. Compared with other optimization algorithms such as AdaGrad, RMSProp, SGDNesterov, and AdaDelta, Adam's optimization algorithm got better results in the multilayer neural network experiments on the Modified National Institute of Standards and Technology (MNIST) images dataset and the CNN on the Canadian Institute for Advanced Research (CIFAR)10 dataset. The steps used when performing weight updates using the Adam optimizer are as follows [28]:

$$g_t = \nabla_\theta f_t(\theta_{t-1}) \tag{9}$$

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * g_t \tag{10}$$

$$v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * g_t^2 \tag{11}$$

Authorized licensed use limited to: Universitas Diponegoro (UNDIP). Downloaded on May 07,2026 at 15:25:51 UTC from IEEE Xplore. Restrictions apply.

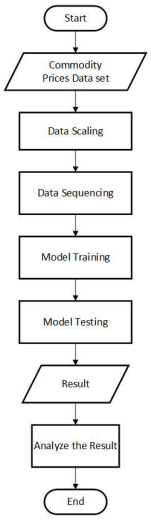


Fig. 5. Research method.

$$\hat{m}_t = m_t / \left(1 - \beta_1^t\right) \tag{12}$$

$$\hat{v}_t = v_t / \left(1 - \beta_2^t\right) \tag{13}$$

$$\theta_t = \theta_{t-1} - \alpha * \hat{m}_t / \left(\sqrt{\hat{v}_t + \epsilon}\right). \tag{14}$$

The first step is calculating the gradient g_t with the previously calculated gradient $\nabla_\theta f_t(\theta_{t-1})$. Then, update the biased estimate of the first moment m_t with b_1 as the first exponential decay, the biased first moment from the previous time step m_{t-1}, and g_t. Then, update the biased estimate of the second moment v_t with b_2 as the second exponential decay, the biased second moment from the previous time step v_{t-1}, and g_t. Then, calculate the bias-corrected first moment $\hat{m}_t$ and the bias-corrected second moment $\hat{v}_t$. Finally, update the parameter θ_t with the old parameter theta$_{t-1}$, learning rate α, $\hat{m}_t$, $\hat{v}_t$, and ϵ as a small number for numerical stability.

F. Mean Absolute Error (MAE)

MAE is a method to evaluate the results of predictions generated by a model. MAE is the absolute average of the resulting errors between the actual and predicted values, and MAE reflects the error in the actual predicted value [12]. The equation that reflects the MAE calculation process as described above is written as follows:

$$\text{MAE} = \frac{1}{N} \sum_{L=1}^{N} \left| \hat{y_L} - y_L \right| \tag{15}$$

where MAE is the result of MAE calculation, N is the amount of train or test sample, $\hat{y}_L$ is the prediction value, and y_L is the true value.

III. Methodology

In this section, the author will explain the flow of the research methods used in this study. In this study, several steps will be carried out. First, the flow of the research methods will be visualized with the flowchart below.

Fig. 5 is the flow of this research method, and the first step is to prepare the commodity prices dataset, scale the dataset, and then divide the data into several sequences. After that, the data will be trained on the model. Then, the model will be tested with the testing dataset. Finally, the results will be obtained from these model tests, which will later be analyzed. Further explanation of the flow of research methods will be explained in several subsections below.

A. Obtaining the Data

The data in this study are gathered from the official government website managed by the Indonesian Ministry of Finance and Bank Indonesia, which provides data on commodity prices in Indonesia. The website name is PIHPS Nasional [18]. It presents food commodities that significantly contribute to the formation of (strategic) inflation rates in Indonesia. The dataset received from websites has the extension of xlsx and is complex in structure. Therefore, to simplify data processing and model training, the authors changed the dataset extension to csv. In this dataset, the recorded commodity prices range from August 1, 2017, to July 30, 2021, with a total of 987 lines. In addition, the dataset contains 11 commodity features, such as rice, chicken meat, egg, onion, garlic, large red chilies, curly red chilies, red chilies, green chilies, cooking oil, and sugar.

B. Data Normalization

Data normalization normalizes the data to have a value according to the specified scale. For example, the obtained dataset lists commodity prices on a large-scale rupiah scale. Large-scale values will make it difficult for the model to find the best parameter values so that the output issued by the model will not be good enough. Therefore, the author will normalize the dataset using a min–max scaler to avoid this. The output of the normalization process is a list of commodity prices with values between 0 and 1.

C. Data Sequencing

Data sequencing is a process of dividing the dataset into several batches of data representing a set at a certain period. In this study, the dataset will be divided into 955 batches, with one batch having a sequence length of 30 data. For example, one batch of data will consist of 30 days of calendar on the dataset and be split into 80% of data training and 20% of data testing. That means that data training will consist of 764 batches, while data testing will consist of 191 batches.

D. Model Training

The normalized training data will undergo a training process on a model created with several epochs and a predetermined optimization function. The optimization functions used in this research are SGD and Adam. In addition, this study will compare the results of SGD and Adam using several LSTM configurations.

E. Model Testing

After conducting model training, the next step is to evaluate the model. This evaluation uses data prepared in advance, with 191 batches. The results generated from the model will be analyzed next.

Authorized licensed use limited to: Universitas Diponegoro (UNDIP). Downloaded on May 07,2026 at 15:25:51 UTC from IEEE Xplore. Restrictions apply.

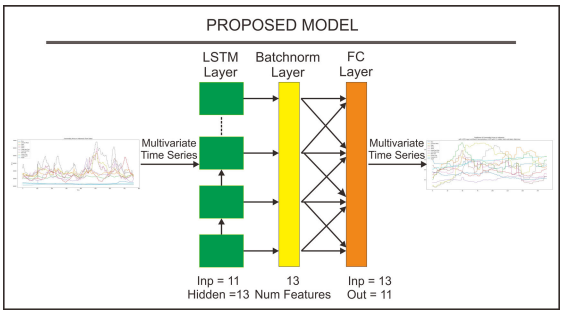


Fig. 6. Proposed model.

F. Analyze the Result

After model testing is complete, an analysis will be carried out on the results generated from model's prediction of the test data. First, the model will be analyzed using the MAE method, calculating the absolute error value between the predicted value and the actual value written in (15). Then, the analysis that will be carried out compares the configuration of the model and the optimizer, which produces the lowest MAE value.

IV. Proposed LSTM Model Configuration

To be able to predict commodity prices, the authors propose the configuration of the LSTM model as depicted in Fig. 6.

As shown in Fig. 6 above, the configuration of the model will consist of three layers, namely, standard LSTM, Batchnorm, and a fully connected (FC) layer with a sigmoid activation function added. Furthermore, the optimizer used in this study is Adam. These layers will be described in the subchapter below.

A. Multivariate LSTM

Some weather, ecology, and economic problems are often related to other variables. In other words, between one variable and another variable has a dependency [29]. For example, in a weather forecast dataset, the variables of temperature, wind speed, humidity, air pressure, and rainfall will be interrelated and depend on each other to predict the weather for that day. To illustrate this, the following is a representation of a dataset that has multivariate features:

$$\hat{x}_{v,t} = f\left(x_{1,t}, x_{v,t}, \ldots, x_{1,t+1}, x_{v,t+1}, \ldots, x_{1,t+n}, x_{v,t+n}\right) \quad (16)$$

where $\hat{x}$ is the prediction from the multivariate dataset, v is the number of variables from the dataset, t is the time step from the dataset, f is a function to predict, and n is the increment of the time step from the dataset.

Fig. 7 shows the multivariate LSTM, where x is the input data, $\hat{x}$ is the output data, v is the number of variables, t is the number of time steps, and T is the number of future time steps. The LSTM deep learning method can predict time series on datasets with more than one variable (multivariate). This certainly has excellent benefits because methods such as classical linear have difficulty modeling multivariate problems [30].

The data to be processed in this study are the temporal data with many features or variables. Thus, the use of the multivariate LSTM model is suitable because, according to the related work discussed in chapter II, it shows that the use of the multivariate LSTM is better for handling multivariate datasets than other methods in previous studies. The multivariate LSTM layer architecture proposed in this study is in the form of one LSTM layer, 11 inputs (according to the number of features used), and 13 hidden units.

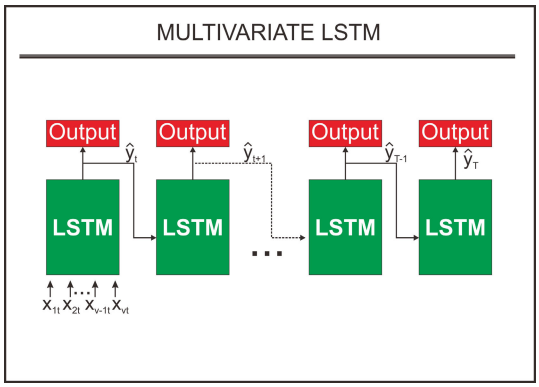


Fig. 7. Multivariate LSTM.

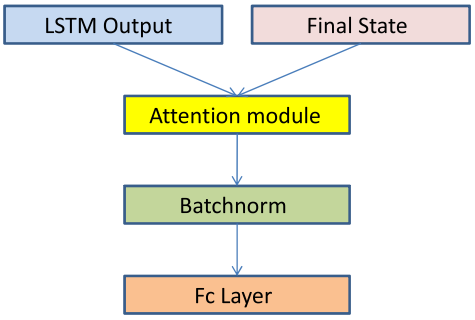


Fig. 8. Attention LSTM.

B. Attention Multivariate LSTM

The attention mechanism is a module in deep networks to force the model to focus on the important features. For sentence translation, the attention module in deep neural nets is introduced by Vaswani et al. [31]. In this research, we attach the attention module to an output of LSTM before Batchnorm and FC layer as shown in Fig. 8.

As shown in Fig. 9, the attention module simply multiplies the LSTM output with the final hidden states to produce attention weight. Then, attention weight is fed into the softmax function in sequencewise and featurewise activation, respectively, to produce soft attention weight. The attention module's output is a context resulting from elementwise multiplication of the LSTM output and soft attention weights. Later, it is fed into Batchnorm and FC layer.

C. Batch Normalization

Because the data to be processed have many features and batches, there is a possibility of an ICS. The Batchnorm layer works by independently standardizing the input data for each scalar feature by making it have a mean of 0 and a variance of 1. The Batchnorm layer has advantages, including the model can use a higher learning rate to reach the convergence point

Authorized licensed use limited to: Universitas Diponegoro (UNDIP). Downloaded on May 07,2026 at 15:25:51 UTC from IEEE Xplore. Restrictions apply.

```
def attention_net(self,lstm_output, final_state):
    # lstm_output : [batch_size, n_step, n_hidden], F matrix
    # final_state : [num_layers(=1), batch_size, n_hidden]

    batch_size = len(lstm_output)
    hidden = final_state[0]
    attn_weights = lstm_output * hidden.unsqueeze(1)
    soft_attn_weights = F.softmax(attn_weights,1)
    soft_attn_weights = F.softmax(soft_attn_weights,2)
    context = lstm_output*soft_attn_weights

    return context, soft_attn_weights
```

Fig. 9. Attention module.

faster without worrying about exploding and vanishing gradients. Furthermore, this layer makes the model more general to minimize or avoid overfitting conditions. The number of features used in this configuration is 13, which indicates the number of the output feature dimensions from the previous LSTM layer.

D. FC Layer

Finally, the data will be processed at the FC layer, the output layer of this model configuration. The input features used in this layer are 13, according to the number of output features from the previous Batchnorm layer. Then, the output features used are 11, according to the number of commodity features in the dataset. This means that in this study, multicommodity prices will be carried out by predicting each feature price. Then, in this layer, the sigmoid activation function will be used to produce an output number between 0 and 1 following normalization of the data, namely, the min–max scaler with a range of 0–1.

V. Results and Analysis

In this chapter, the author will explain the results and analysis obtained from testing model implementation described in Section III. The tests include the comparison of several LSTM model configurations, the effect of adding a Batchnorm layer, testing several hyperparameters' configurations, and testing the reduction in the number of commodity price features. The author will start by explaining the test configurations.

A. Train and Test Configurations

The author uses the PyTorch framework to train and build the model. Then, to optimize the model, the author will compare two optimizers, Adam and SGD. In addition, the epoch used in the training process is 1500, with a sequence length of 30 data. The initial learning rate value used is 0.1 with a learning rate scheduler of ReduceLROnPlateau with patience of 100. In addition, the author uses a seed of 23 to be used for torch manual seed and NumPy random seed.

B. Testing the Model Configuration on the Loss Value

Testing the model configuration on the loss in the form of MAE is carried out to compare the loss of the four configurations, namely, LSTM, bidirectional LSTM, convolution LSTM, and convolution bidirectional LSTM. The best model configuration is the model with the lowest loss in terms of MAE.

TABLE I
Results of Model Configuration Test

Evaluation	Optimizer	MAE Train	MAE Val	MAE Test	MAE Test Denormalized
LSTM	Adam	0.01906	0.10263	**0.13379**	**2615.05528**
	SGD	0.03199	0.11693	0.16712	3851.35866
Bi-LSTM	Adam	0.10486	0.1424	0.16093	3524.88741
	SGD	0.03237	0.13795	0.1975	4185.47941
Conv LSTM	Adam	0.02874	**0.09688**	0.14296	3641.11582
	SGD	0.03482	0.12148	0.15274	3570.88937
Conv Bi-LSTM	Adam	**0.01181**	0.11698	0.14327	3078.36087
	SGD	0.03621	0.12063	0.15105	3506.2

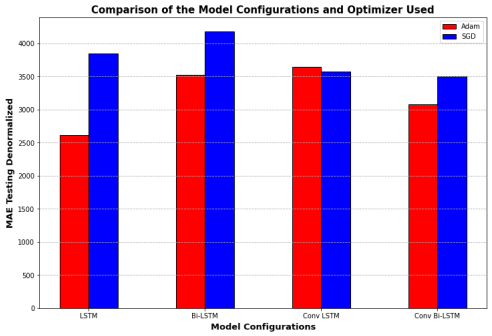


Fig. 10. Comparison of the model configurations and optimizer used.

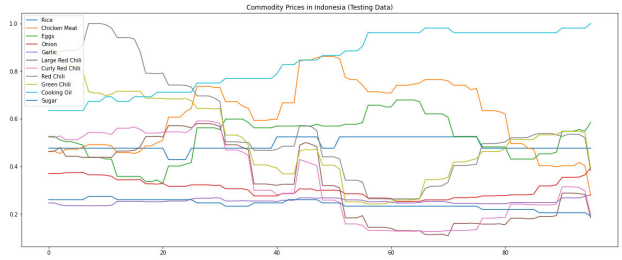


Fig. 11. Commodity prices (testing data).

Table I displays the results of testing the model configuration on the loss value (lower is better). Fig. 10 shows comparison of the MAE denormalized testing data and optimizer. From the table and figure, it can be concluded that the best model configuration is the standard LSTM using the Adam optimizer with an MAE of 2615.05528. The results obtained from this standard LSTM configuration are better than the bidirectional LSTM or when a convolution layer is added. This is because the dataset used has less meaning when the sequences used are reversed, or in other words, the values that exist in the future do not affect the values that existed in the past, so the use of bidirectional LSTM has a higher loss value than the standard LSTM. In addition, using the convolution layer also has results that are inferior to the standard LSTM. The dataset used only has a temporal and no spatial aspect, so adding a convolution layer is insignificant.

The Adam optimizer was also found to have a lower loss value than the SGD optimizer. This is because the Adam optimizer has the advantages of AdaGrad and RMSProp. In addition, Adam can work well with sparse gradients (advantages of AdaGrad) and adjust the weights to the latest average gradient magnitude, which means it can work well in nonstationary conditions (noisy).

Fig. 11 depicts the test data on commodity prices, while Fig. 12 shows the predicted results of the standard LSTM

Authorized licensed use limited to: Universitas Diponegoro (UNDIP). Downloaded on May 07,2026 at 15:25:51 UTC from IEEE Xplore. Restrictions apply.

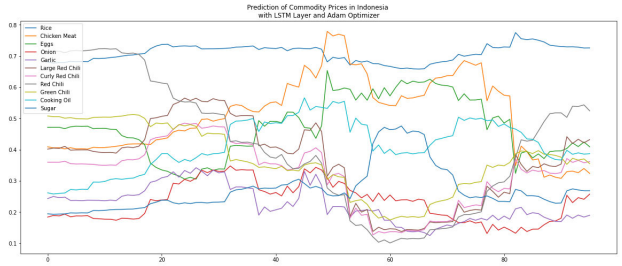


Fig. 12. Commodity prices' prediction with the LSTM layer and Adam optimizer.

TABLE II
RESULTS OF ADDING BATCHNORM LAYER TEST

Evaluation	MAE Train	MAE Val	MAE Test	MAE Test Denormalized
LSTM	0.01906	0.10263	0.13379	2615.05528
Batchnorm LSTM	**0.01087**	**0.0353**	**0.0731**	**1786.53447**

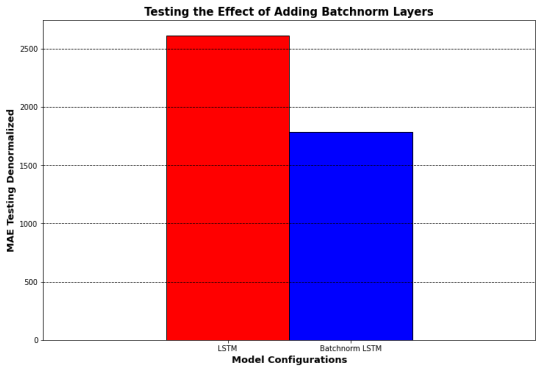


Fig. 13. Comparison of the non-Batchnorm LSTM and Batchnorm LSTM.

configuration using the Adam optimizer. In this configuration, it can be seen that the prediction graphs are slightly similar to Fig. 11. The difference is that the resulting graphs are noisy and spiky in Fig. 12.

C. Testing Effect of Adding a Batchnorm Layer

In this section, we investigate the effect of adding a layer of Batchnorm to the LSTM. Because in the previous test, the configuration of the standard LSTM model with the Adam optimizer was the model configuration with the lowest loss. The standard LSTM model without and with batch norm using the Adam optimizer is evaluated in this comparison. Therefore, the configuration of the model used in this test is as follows.

1) Standard LSTM (LSTM → LSTM → Linear → Sigmoid).
2) Standard LSTM (LSTM → LSTM → Batchnorm → Linear → Sigmoid).

Table II displays the results of testing the effect of adding a batch norm layer to LSTM. Concurrently, Fig. 13 shows the visual comparison of MAE on the denormalized test data on the two models. From the table and figure, it can be seen that the addition of the batch norm layer dramatically affects the loss. The loss value in the training and testing phases decreased significantly when the batch norm was added to the model configuration. In addition, the difference between the MAE training and MAE testing values in the model configuration with the addition of a batch norm layer indicates that the overfitting condition can be reduced by adding a batch norm layer to the model configuration.

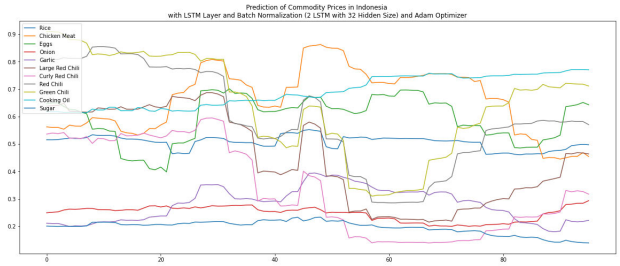


Fig. 14. Commodity prices' prediction with the Batchnorm LSTM layer.

TABLE III
RESULTS OF NUMBER OF LAYERS' TEST

Evaluation	MAE Train	MAE Val	MAE Test	MAE Test Denormalized
1 Layer	**0.0107**	**0.03169**	**0.06513**	**1513.09104**
2 Layers	0.01087	0.0353	0.0731	1786.53447
3 Layers	0.01897	0.08476	0.11476	2072.72553
4 Layers	0.10421	0.17186	0.23472	6378.37821

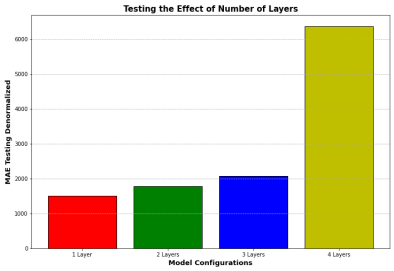


Fig. 15. Comparison of the number of layers used.

Fig. 14 shows the prediction results of the standard LSTM configuration using Batchnorm. In Fig. 12, the resulting graph only has a slight similarity to Fig. 11, which shows a graph of test data from a previously provided dataset. Then in Fig. 14, the resulting graph is close and begins to show similarities to Fig. 11, which shows the ground truth of the testing data. Adding a Batchnorm layer can reduce overfitting and improve the resulting model.

D. Testing the Model Hyperparameter Configuration

The next test is a hyperparameter evaluation of the model on LSTM. Hyperparameter testing that will be carried out is changing the number of LSTM layers and hidden sizes.

1) Testing the Effect of Number of Layers: The number of LSTM layers will be evaluated against the loss value obtained in this test. The model's configuration used is the Batchnorm Standard LSTM with the Adam optimizer because this model has the best evaluation result (lowest MAE) based on the previous test. The number of LSTM layers to be tested includes one LSTM layer, two LSTM layers, three LSTM layers, and four LSTM layers. The number of hidden sizes is set to 32.

Table III shows the results of the number of layers' test, and Fig. 15 is the visual comparison between the number of layers used in this test. In the table and figure, it is clear that the configuration with only one layer of LSTM outperforms the other configurations. One layer of Batchnorm Standard LSTM has a lower train, validation, test, and denormalized

Authorized licensed use limited to: Universitas Diponegoro (UNDIP). Downloaded on May 07,2026 at 15:25:51 UTC from IEEE Xplore. Restrictions apply.

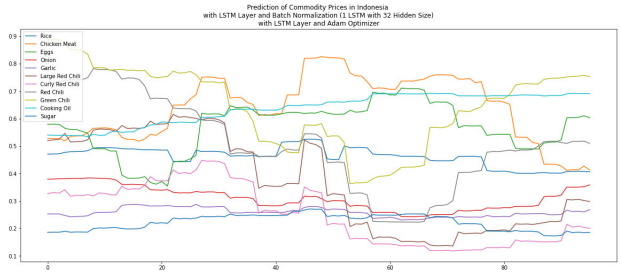


Fig. 16. Commodity prices prediction with Batchnorm LSTM (1 layer).

TABLE IV
RESULTS OF NUMBER OF HIDDEN SIZE TEST

Evaluation	MAE Train	MAE Val	MAE Test	MAE Test Denormalized
11 Hidden	0.01162	0.03833	0.07611	1661.7363
12 Hidden	0.012	0.0318	**0.04836**	991.35123
13 Hidden	0.01107	0.02512	0.05079	**988.66981**
14 Hidden	**0.01102**	0.0281	0.07199	1751.60891
15 Hidden	0.01165	0.02909	0.05466	1129.32576

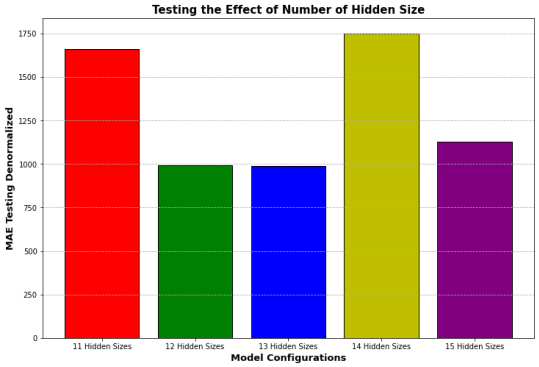


Fig. 17. Comparison of the number of hidden sizes used.

test MAE values than the other configurations. This, of course, has the advantage of having the lowest computation time while keeping prediction results at 1513.09104.

Fig. 16 shows the prediction results of the Batchnorm Standard LSTM configuration with one LSTM layer and 32 hidden sizes. In the graph, the predictions generate similar patterns to the ground-truth data in Fig. 11, which means the use of one layer of LSTM has good results than two, three, or four layers of LSTM. The color of the line on each graph represents the prediction of each commodity in the dataset.

2) Testing the Effect of the Number of Hidden Size: In this test, an evaluation of the number of hidden sizes used in the LSTM and linear layers will be evaluated in terms of loss value. The selected model contains the Batchnorm Standard LSTM with one layer of LSTM because this configuration produced the best results based on the previous test. Furthermore, because the number of features contained in the dataset is 11, we use the number of hidden sizes close to the number of features, 11–15 of hidden sizes.

Table IV is a table that contains information about the effect of the number of hidden sizes on the loss value. Fig. 17 shows a graph of the comparison of MAE obtained from various hidden sizes used in the configuration. The best result obtained in this test is the Batchnorm Standard LSTM configuration with a hidden size of 13 hidden sizes with an MAE on denormalized test data of 988.66981. However, this result is the lowest obtained compared with other configurations.

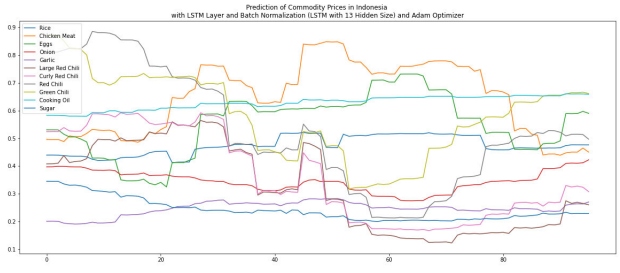


Fig. 18. Commodity prices' prediction with Batchnorm LSTM (1 layer and 13 hidden size).

TABLE V
RESULTS OF REDUCTION FEATURES' TEST

Without	MAE Train	MAE Val	MAE Test	MAE Test Denormalized
Onions & Garlic	0.01149	0.03029	0.07275	1389.85058
All Chilies	**0.00959**	**0.01494**	0.04644	**267.34908**
Chicken & Egg	0.01015	0.02503	0.05493	1268.67605
Rice, Oil, Sugar	0.01175	0.02248	**0.02774**	1056.19676

Fig. 18 shows prediction results of using the Batchnorm Standard on LSTM during training with the configuration of one LSTM layer and 13 hidden units. In this configuration, the model produces the lowest MAE of denormalized testing data, and thus the resulting graphs are also closest to the ground truth in Fig. 11. The color of the line on each graph represents the prediction of each commodity.

E. Testing the Reduction in the Number of Commodity Prices Features

The next test is carried out by reducing the number of features included in the multivariate LSTM. The selected model contains the Batchnorm and standard LSTM model with one LSTM layer, 13 hidden sizes, and the use of Adam optimizer. This model configuration has the best performance compared with other models based on the previous result. If all the previous evaluations use 11 features (according to the number of commodity prices contained in the dataset), then in this evaluation, several configurations will be applied, which are as follows.

1) Testing without features of onions and garlic (nine features).
2) Tests without features of large red chili, curly red chili, red chili, and green chili (seven features).
3) Testing without features of chicken meat and eggs (nine features).
4) Testing without features of rice, cooking oil, and sugar (eight features).

Table V displays the effect of the number of features on the loss value obtained, and Fig. 19 shows the comparison between the number of features used in this test. For example, in this test, the configuration without large red chilies, curly red chilies, red chilies, and green chilies produces the lowest MAE. This indicates that the features of large red chilies, curly red chilies, red chilies, and green chilies have little effect on other features. When these features are excluded, the model can be trained more effectively to obtain the lowest MAE up to 267.34908.

Fig. 20 shows test data on commodity prices' dataset in Indonesia without features of all chilies. Fig. 21 shows the prediction results using the Batchnorm Standard LSTM model

Authorized licensed use limited to: Universitas Diponegoro (UNDIP). Downloaded on May 07,2026 at 15:25:51 UTC from IEEE Xplore. Restrictions apply.

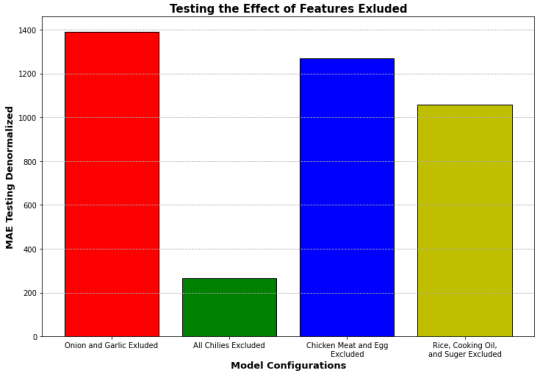


Fig. 19. Comparison of the features' reduction used.

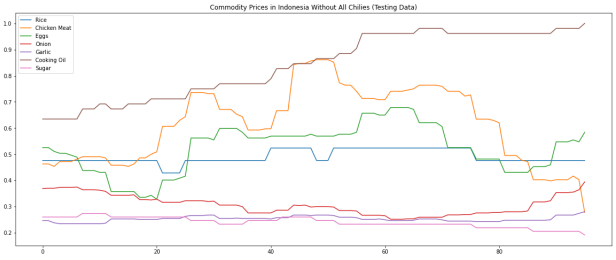


Fig. 20. Price of commodities in Indonesia without all chilies.

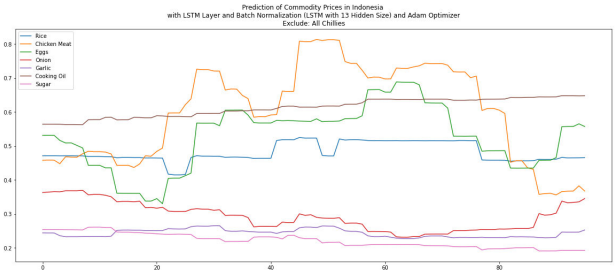


Fig. 21. Commodity prices' prediction without all chilies using Batchnorm LSTM (1 layer and 13 hidden sizes).

with one layer of LSTM and 13 hidden sizes. If the two graphs are compared, the brown colored line is seen as different from the ground truth, namely, the feature of cooking oil. Other than that, all the lines look similar and correspond to their ground truth (Fig. 20). In this test, the lowest MAE value was also achieved in the denormalized test data, 267.34908 of MAE. The color of the line on each graph represents the prediction of each commodity in the data.

F. Correlation Analysis

Next, we will figure out the correlation of each feature from the dataset. Here is the figure that shows the correlations of each pair of attributes.

Fig. 22 shows feature pairs' correlations, which means how much one variable correlates with another. According to [32], ± 1.0 indicates a perfect positive/negative correlation, the range of ± 0.5 and ± 1.0 shows a high degree positive/negative correlation, the range of ± 0.3 and ± 0.49 represents a moderate positive/negative correlation, at below ± 0.29 suggests a low degree positive/negative correlation, and 0.0 indicates a zero correlation. From Fig. 22 above, we can conclude that all chili peppers have a high degree of correlation with each other. Therefore, these features can become one multivariate

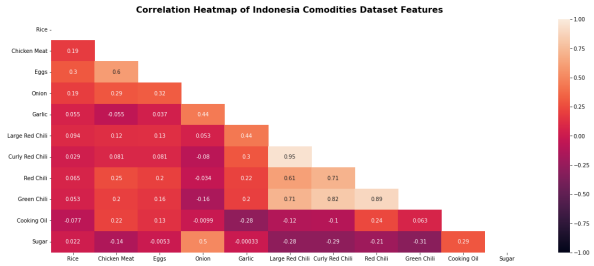


Fig. 22. Commodity prices' correlations.

TABLE VI
RESULTS OF THE NUMBER OF LAYERS' TEST ON THE TEN-DAY PREDICTION

Evaluation	MAE Train	MAE Val	MAE Test	MAE Test Denormalized
1 Layer	0.03825	0.13997	0.19034	5242.75605
2 Layers	**0.0367**	**0.12477**	**0.1654**	**4498.42131**
3 Layers	0.04854	0.15737	0.19224	5341.24392
4 Layers	0.14509	0.1685	0.20225	5787.66685

group because of the high correlation. In addition, chicken meat–eggs and onion–eggs are also categorized as feature pairs with a high correlation, even though not as high as the chilies group. So, chicken meat, eggs, and onion can also become one multivariate group. The figure also shows that there is a correlation occurring between other variables with a low to a medium degree.

G. Testing the Longer Prediction

We also experiment with how the capability of the network model can predict the longer sequence. As shown in Table VI, the lowest accuracy is obtained by the two-layer LSTM for the ten-day prediction. However, more stacked LSTM starts overfitting. While the rest of the experiment is carried out on a one-day forecast, this proves that as the problem becomes complex, stacked LSTM can be a solution.

We used 13 hidden sizes with Batchnorm and Adam optimizer during training. For this experiment, all the features are included as input of the multivariate LSTM.

H. Testing the Comparison of Attention Multivariate LSTM, Multivariate LSTM and Univariate LSTM

After that, we conduct a comparison of the multivariate and univariate LSTM. The configuration model used in this evaluation is the Batchnorm Standard LSTM model with one LSTM layer, 13 hidden sizes, and the Adam optimizer because that configuration has the best performance compared with other models on a one-day prediction. Note that for multivariate LSTM, we exclude sigmoid activation on the last layer due to the limitation of data. Then, the features used are grouped based on correlation analysis in Fig. 22.

Table VII displays the comparison test results between the attention multivariate LSTM, multivariate, and univariate LSTM. In this test, the average MAE of the grouped features of the attention multivariate LSTM model has the lowest MAE than the others. Moreover, the multivariate LSTM got better MAE on average than the univariate one. This indicates that these grouped features correlate with each other and are thus beneficial for prediction.

Authorized licensed use limited to: Universitas Diponegoro (UNDIP). Downloaded on May 07,2026 at 15:25:51 UTC from IEEE Xplore. Restrictions apply.

TABLE VII

COMPARISON TEST RESULTS OF MULTIVARIATE LSTM WITH NON-MULTIVARIATE LSTM IN MAE AFTER DENORMALIZATION

Group Features	Attention Multivariate LSTM	Multivariate LSTM	Univariate LSTM
Chicken	174.499	**174.305**	174.534
Egg	80.618	**80.058**	82.523
Onion	**54.378**	67.967	56.922
Onion	53.631	**53.116**	56.922
Garlic	129.476	**128.755**	280.598
Onion	**52.365**	52.984	56.922
Sugar	6.066	**5.894**	6.923
Large Red Chili	550.03	596.728	**545.05**
Curly Red Chili	566.106	585.497	**559.996**
Red Chili	**1151.878**	1285.248	2110.032
Green Chili	**491.966**	524.214	493.146
Mean	**255.998**	323.160	402.142

TABLE VIII

COMPARISON WITH OTHER METHODS' RESULT

Evaluation	MAE Train	MAE Val	MAE Test	MAE Test Denormalized
LSTM	**0.00959**	**0.01494**	0.04644	**267.34908**
RNN	0.01133	0.01843	0.0513	284.11237
CNN	0.01941	0.02176	**0.04012**	564.90985
Transformer	0.03604	0.07707	0.09385	1085.73133

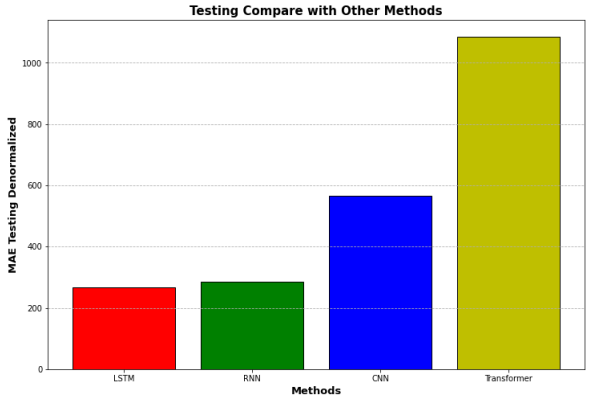


Fig. 23. Comparison with other methods.

I. Comparison With Other Methods

The last test we have done is to compare the proposed method with several other methods. The comparison methods are RNN, CNN, and Transformer. The configuration of each method is as follows.

1) *LSTM (Proposed Method):* LSTM (input size = 7, hidden size = 13) → Batchnorm (number features = 13) → FC layer (input features = 13, out features = 7) → sigmoid.
2) *RNN:* RNN (input size = 7, hidden size = 13) → Batchnorm (number features = 13) → FC layer (input features = 13 and output features = 7) → sigmoid.
3) *CNN:* Convolution1-D (input channels = 7, output channels = 13, kernel size = (13,), padding = 3, padding mode = "replicate") → FC layer (input features = 13 and output features = 7) → sigmoid.
4) *Transformers:* Transformer encoder (d_model = 7, nhead = 7, dim_feedforward = 13) → FC layer (input features = 13 and out features = 7) → sigmoid.

Table VIII displays the effect of the methods used on the loss value obtained, and Fig. 23 shows the comparison between each method test denormalized loss in this test. From the table and figure, it shows that the proposed method (LSTM with Batchnorm) produces the lowest MAE on the denormalized test data. The second lowest MAE on the denormalized test data is RNN with a slight difference when compared with the proposed model. The third and last are the CNN and Transformer models. These results indicate that the proposed method is good in the case of multivariate time series forecasting and the commodity prices' datasets in Indonesia.

VI. CONCLUSION

Based on the results, the best configuration and hyperparameter to produce the lowest possible MAE on Indonesia's commodity price is the use of Batchnorm, an LSTM layer, and 13 hidden sizes. Furthermore, the lowest MAE value was obtained with the features without red chilies, curly red chilies, red chilies, and green chilies during the training and testing phases with an MAE of only 267.34908. The configuration of the multivariate LSTM model is also able to outperform the non-multivariate LSTM model. Moreover, the proposed method gains the lowest MAE on the denormalized test data compared with RNN, CNN, and Transformer.

The results of this study can be used by several groups, especially MSME business people who sell commodity materials and their derivative products when the world's macroeconomic conditions are uncertain due to several global events, such as the war and the ongoing Covid-19 outbreak. With the results of this study, MSME business people can perform preventive actions if the commodity they sell is predicted to experience price spikes. For example, they are scheduling the right time to stock up on cooking oil more than usual because cooking oil prices are predicted to be higher than usual, so MSME business people get higher profits. In addition, MSME business people can also be grouped based on the commodities they sell, such as chili and poultry sellers with chicken products that have a high correlation. The aim is to provide insight into good planning regarding stock and prices in the market. Of course, not only can MSME business people benefit from this study. The government also can design national economic planning in the event of a drastic increase or decrease in commodity prices so that they can control commodity prices and the national economic situation returns to normal.

For further studies, LSTM can be used on the time series dataset in the form of prices because it has the lowest loss value than the bidirectional LSTM and convolution LSTM. After all, time series dataset in the form of prices does not provide relevant information when the sequence is reversed and has no spatial aspect. Then, adding a layer of Batchnorm can help reduce overfitting. Furthermore, the depth of an LSTM model can be added as long as the training results from the model do not experience overfitting. The use of the Adam optimizer produces a lower MAE than SGD because Adam has the advantage of adjusting the weights to the average of the latest gradient magnitude and on the sparse gradient state. In addition, further studies can use an updated dataset because commodity price data in Indonesia on the National PIHPS are continuously updated. Moreover, adding a dropout layer can be applied to the LSTM layer so that the state of overfitting can be reduced. Finally, other deep learning methods, such as the gated recurrent unit (GRU), can be compared and evaluated in future works.

REFERENCES

[1] N. Sutjipto, *Ketahanan UMKM Jawa Timur Melintasi Pandemi COVID-19*. Yogyakarta, Indonesia: K-Media, 2020.

Authorized licensed use limited to: Universitas Diponegoro (UNDIP). Downloaded on May 07,2026 at 15:25:51 UTC from IEEE Xplore. Restrictions apply.

[2] S. M. Jannah and V. F. Thomas. (2021). *Harga Pangan Naik, Umkm Menjerit, Masyarakat Semakin Kesusahan*. Accessed: Sep. 21, 2021. [Online]. Available: https://tirto.id/hargapangan-naik-umkm-menjerit-masyarakat-semakin-kesusahan-f85f
[3] N. Yudistira, "COVID-19 growth prediction using multivariate long short term memory," *IAENG Int. J. Comput. Sci.*, vol. 47, no. 4, pp. 1–8, 2020.
[4] S. Siami-Namini, N. Tavakoli, and A. S. Namin, "A comparison of ARIMA and LSTM in forecasting time series," in *Proc. 17th IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, Dec. 2018, pp. 1394–1401.
[5] N. Selle, N. Yudistira, and C. Dewi, "Perbandingan prediksi penggunaan listrik dengan menggunakan metode long short term memory (LSTM) dan recurrent neural network (RNN)," *Jurnal Teknologi Informasi dan Ilmu Komputer*, vol. 9, no. 1, pp. 155–162, 2022.
[6] N. Wu, B. Green, X. Ben, and S. O'Banion, "Deep transformer models for time series forecasting: The influenza prevalence case," 2020, *arXiv:2001.08317*.
[7] Z. Zhao et al., "LSTM network: A deep learning approach for short-term traffic forecast," *IET Intell. Transp. Syst.*, vol. 11, no. 2, pp. 68–75, 2017.
[8] N. Yudistira, S. B. Sumitro, A. Nahas, and N. F. Riama, "Learning where to look for COVID-19 growth: Multivariate analysis of COVID-19 cases over time using explainable convolution–LSTM," *Appl. Soft Comput.*, vol. 109, Sep. 2021, Art. no. 107469. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1568494621003926
[9] W. Makhlouf and R. Malhotra, "Stock market price prediction using long short-term memory (LSTM)," *Int. J. Adv. Res. Eng. Technol.*, vol. 11, no. 10, pp. 1692–1707, 2020.
[10] X. Song et al., "Time-series well performance prediction based on long short-term memory (LSTM) neural network model," *J. Petroleum Sci. Eng.*, vol. 186, Mar. 2019, Art. no. 106682.
[11] T. Fischer and C. Krauss, "Deep learning with long short-term memory networks for financial market predictions," *Eur. J. Oper. Res.*, vol. 270, no. 2, pp. 654–669, 2018.
[12] C. Tian, J. Ma, C. Zhang, and P. Zhan, "A deep neural network model for short-term load forecast based on long short-term memory network and convolutional neural network," *Energies*, vol. 11, no. 12, p. 3493, Dec. 2018.
[13] L. Xingwei and S. Kuniaki, "The comparison between ARIMA and long short-term memory for highway traffic flow prediction," *J. Eastern Asia Soc. Transporation Stud.*, vol. 13, pp. 1817–1834, 2019.
[14] A. Arfan and L. Etp, "Perbandingan algoritma long short-term memory dengan SVR pada prediksi Harga Saham di Indonesia," *PETIR*, vol. 13, no. 1, pp. 33–43, Mar. 2020.
[15] A. Biswaal. (2021). *A Guide to Time Series Forecasting in R You Should Know*. accessed: Aug. 23, 2021. [Online]. Available: https://www.simplilearn.com/tutorials/data-science-tutorial/time-series-forecasting-in-r
[16] Novalia. (2018). *Faktor Yang Mempengaruhi Harga Komoditas Di Indonesia*. Accessed: Sep. 21, 2021. [Online]. Available: https://www.seputarforex.com/artikel/faktor-yang-mempengaruhi-harga-komoditas-di-indonesia-286323-33
[17] S. K. Dermoredjo, Y. H. Saputra, and D. H. Azahari, "Dampak pandemi COVID-19 terhadap perdagangan dalam Negeri komoditas pertanian," in *Dampak Pandemi COVID-19, Perspektif Adaptasi dan Resiliensi Sosial Ekonomi Pertanian*, A. Suryana, I. W. Rusastra, T. Sudaryanto, and S. M. Pasaribu, Eds. Jakarta, Indonesia: IAARD Press, 2020, pp. 127–148.
[18] P. Nasional. (2021). *Pusat Informasi Harga Pangan Strategis Nasional*. Accessed: Sep. 21, 2021. [Online]. Available: https://hargapangan.id/tabel-harga/pedagang-besar/komoditas
[19] F. Chollet, *Deep Learning With Python*. Shelter Island, NY, USA: Manning Publications, 2018.
[20] K. Wang, X. Qi, and H. Liu, "Photovoltaic power forecasting based LSTM-convolutional network," *Energy*, vol. 189, Dec. 2019, Art. no. 116225.
[21] Pytorch. (2019). *LSTM*. Accessed: Oct. 15, 2021. [Online]. Available: https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html
[22] J. Brownlee, *Long Short-Term Memory Networks With Python: Develop Sequence Prediction Models With Deep Learning*. Machine Learning Mastery, 2017. [Online]. Available: https://machinelearningmastery.com/
[23] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," 2015, *arXiv:1502.03167*.
[24] Pytorch. (2019). *Batchnorm1d*. Accessed: Nov. 11, 2021. [Online]. Available: https://pytorch.org/docs/stable/generated/torch.nn.BatchNorm1d.html
[25] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
[26] A. Ai. (2020). *Stochastic Gradient Descent*. Accessed: Aug. 23, 2021. [Online]. Available: https://www.akira.ai/glossary/stochastic-gradient-descent/
[27] K.-L. Lim, "Non-conjugate posterior using stochastic gradient ascent with adaptive stepsize," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2020, pp. 983–992.
[28] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv*, 2017.
[29] H. Widiputra, A. Mailangkay, and E. Gautama, "Multivariate CNN-LSTM model for multiple parallel financial time-series prediction," *Complexity*, vol. 2021, Oct. 2021.
[30] J. Brownlee. (2021). *Multivariate Time Series Forecasting With LSTMS in KERAS*. Accessed: Dec. 17, 2021. [Online]. Available: https://machinelearningmastery.com/multivariate-time-series-forecasting-lstms-keras/
[31] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–11.
[32] S. Glen. (2021). *Correlation Coefficient: Simple Definition, Formula, Easy Steps*. Accessed: Aug. 3, 2020. [Online]. Available: https://www.statisticshowto.com/probability-and-statistics/correlation-coefficient-formula/



Marsellino Prawiro Halim received the Bachelor of Computer Science degree from the University of Brawijaya, Malang, Indonesia, in 2022.

He is currently a Campaign Analytic at Bank Mandiri, Indonesia. He is currently interested in the personalized campaign using machine learning, deep learning, and big data analysis for analytics about customer behavior, and a recommender system for the best product for each customer.



Novanto Yudistira received the B.S. degree in informatics engineering from the Institut Teknologi Sepuluh Nopember, Surabaya, Indonesia, in November in 2007, the M.S. degree in computer science from the Universiti Teknologi Malaysia, Johor Bahru, Malaysia, in 2011, and the Dr.Eng. degree in information engineering from Hiroshima University, Higashihiroshima, Japan, in 2018.

In 2016, he was involved in research collaboration with the Mathematical Neuroinformatics Group, National Institute of Advanced Industrial Science and Technology (AIST), Tokyo, Japan. In 2018, he continued as a Post-Doctoral Fellow in informatics and data science analytic for two years working with the Japanese large scientific research institute, RIKEN, Tokyo, and Osaka University, Suita, Japan. He is currently a Lecturer and a Researcher at Brawijaya University, Malang, Indonesia. His current research interests include deep learning, multimodal computer vision, medical informatics, and big data analysis.



Candra Dewi received the B.Sc. degree in informatics from the Institut Teknologi Sepuluh Nopember, Surabaya, Indonesia, in 2001, and the master's degree in information technology for natural resource management from the Bogor Agricultural Institute, Bogor, Indonesia, in 2008. She is currently pursuing the Ph.D. degree with the Department of Biology, University of Brawijaya, Malang, Indonesia, in the field of computational biology using computer vision and machine learning.

Since 2012, she has been with the Intelligent Computing Research Group, Faculty of Computer Science, Brawijaya. Her research focus is on implementation of machine learning for various cases (i.e., pattern recognition, optimization, etc.) and image processing and computer vision for various images (i.e., camera/mobile camera image, satellite image, aerial photo image captured using drone/unmanned aerial vehicle (UAV), microscope image, etc.)

Authorized licensed use limited to: Universitas Diponegoro (UNDIP). Downloaded on May 07,2026 at 15:25:51 UTC from IEEE Xplore. Restrictions apply.